# Calcul de requêtes acycliques conjonctives dans une base de données

Nathan Tillier

31 mai 2026

## Introduction

Les bases de données relationnelles permettent de structurer et d'interroger de grandes quantités de données. Cependant, le calcul de requêtes conjonctives se heurte à une barrière majeure : il s'agit d'un problème NP-complet.
Toutefois, un certain nombre de requêtes, celles conjonctives acycliques, peuvent être évaluée en temps linéaire grâce à l'algorithme de Yannakakis. À travers l'implémentation d'un moteur de requête en Python et son application à un jeu de données réelles (le découpage administratif français en communes, départements et régions), nous nous intéresserons au fonctionnement de cet algorithme ainsi qu'aux résultats expérimentaux qu'il permet.

## 1 Algèbre relationnelle

### 1.1 Notations

Une relation $R$ sera notée comme suit :

$$\mathrm{R}(\mathrm{attribut}_1 : \mathrm{domaine}_1, ..., \mathrm{attribut}_n : \mathrm{domaine}_n)$$

Sa création correspond au code SQL ci-dessous

```sql
CREATE TABLE R(
    domaine_1 attribut_1,
    ...
    domaine_n attribut_n
)
```

Par exemple,

```sql
CREATE TABLE commune (
    INT id PRIMARY KEY,
    INT departement,
    CHAR nom
)
```

Une instance $I$ sur une relation $R$ est un ensemble fini de n-uplets (ou tuples) sur $R$, qu'on notera $R^I$.
Une instance $I$ sur le schéma d'une base de données est l'union des instances des relations $R$ du schéma.

On utilisera les opérations suivantes :

- $\pi_V(R)$ la projection d'une relation $R$ sur un ensemble de variables $V$ : elle renvoie la table contenant les tuples distincts de $R$ dont on n'a conservé que les attributs présents dans $V$
- $R \bowtie S$ la jointure de $R$ et $S$ : il s'agit des combinaisons de tuples de $R$ et $S$ ayant un attribut commun.

## 1.2 Requêtes conjonctives

**Définition** (Requête conjonctive)**.** *On considère des requêtes conjonctives sous la forme*

$$q(x_1, ..., x_n) : -R_1(\vec{y_1}), ..., R_p(\vec{y_p})$$

*Où chaque $y_i$ est un tuple de variables de même arité que $R_i$, et où chaque $x_j$ apparaît dans un $y_i$ au moins. On appellera la partie avant les deux points l'en-tête de la requête et celle après le corps de la requête, chaque expression $R_i(\vec{y_i})$ sera nommée atome.*

On montre ensuite quelques résultats sur les requêtes conjonctives.

**Théorème** (NP-complétude)**.** *L'évaluation d'une requête conjonctive est un problème NP-complet.*

**Démonstration.**

1. *Montrons la NP-difficulté par réduction de 3-SAT.*

*Avec $\varphi$ constituée des variables $x_1, ..., x_k$, on pose pour tout $i$ : ${x_i}' \equiv \neg x_i$.*
*On construit ensuite*

$$I = \{\text{Clause}(0,0,1), ..., \text{Clause}(1,1,1)\} \cup \{\text{Negation}(0,1), \text{Negation}(1,0)\}$$

*de sorte que les tuples de* Clause *représentent les combinaisons de valeurs satisfaisant une clause (au moins un littéral à vrai) et ceux de* Negation *forcent la cohérence entre une variable et sa négation.*
*On pose*

$$q(x_1, ..., x_k) : -\big\{\text{Clause}(x_i, x_j, x_k) \ \text{ ; pour tout } \ (i \vee j \vee k) \ \text{ dans } \ \varphi\big\}, \{\text{Negation}(x_i, {x_i}'); \forall i \in [\![1, k]\!]\}$$

*On remarque qu'il existe une valuation satisfaisant $\varphi$ si et seulement si le résultat de la requête $q(I)$ est non vide.*

2. *Montrons l'appartenance à NP.*

*Étant donné une requête $q$ et une instance $I$, on prend comme certificat l'ensemble des assignations des variables $x_i$ satisfaisant la requête (de taille polynomiale en $|q|$ et $|\text{OUT}|$). Le vérificateur va ensuite vérifier que chaque atome de $q$ existe dans $I$ pour chaque assignation (polynomial en $|q|$, $|I|$ et $|\text{OUT}|$).*

*On conclut que l'évaluation d'une requête conjonctive est NP-complète.*

On s'intéressera donc par la suite à une classe plus restreinte des requêtes conjonctives : celles acycliques.

## 1.3 Requêtes acycliques conjonctives

Intuitivement, une requête acyclique conjonctive est acyclique si les variables communes entre différents atomes « n'introduisent pas de boucle ».
La requête suivante est par exemple cyclique.

```sql
SELECT c.nom, d.nom, r.nom
FROM commune AS c
```

```
3  JOIN departement AS d ON c.departement = d.id
4  JOIN region AS r ON d.region = r.id AND c.region = r.id
```

Car on a la dépendance

commune → commune.departement → departement → departement.region → region → region.id → commune

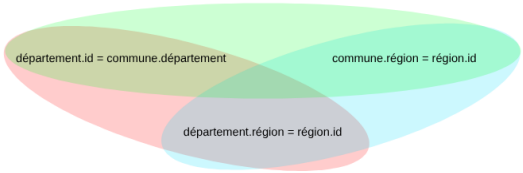



**Abb. 1:** Schéma représentant les dépendances d'une requête cyclique

Formellement, on introduit l'hypergraphe associé à la requête, ayant pour sommets les variables de la requête, et pour arête les ensembles de variables présents dans chaque atome.
Puis avec la réduction Graham-Yu-Ozsoyoglu (GYO), on peut construire un arbre de jointure associé à cet hypergraphe si et seulement si la requête est acyclique. [1]

Cet arbre de jointure vérifie les points suivants :

- ses sommets sont les relations de la requête
- le sous-graphe induit par les relations contenant une même variable partagée est connexe

Sa construction est détaillée dans l'annexe, ainsi que l'algorithme permettant de l'obtenir.

# 2 Calcul du résultat

## 2.1 Implémentation de la base de données

La structure implémentant la logique d'une base de données est une classe `Table` possédant trois attributs (`name`, `rows` et `columns`).

Les champs `name` et `columns` servent à conserver respectivement les noms et attributs des tables, tandis que `rows` est une liste de tuples (immutables en Python) listant les éléments présents dans la table. Dans la mesure où l'on s'intéresse exclusivement à l'évaluation de requêtes, seulement deux méthodes importent : `Table.import_from` permettant de charger un fichier CSV valide, et `Table.project` réalisant une projection sur l'ensemble $V$ des attributs passés en argument.

La construction des requêtes se fait ensuite avec des classes intermédiaires, `Query`, `Head` et `Atom`. La définition d'une requête devient ainsi

```py
Query(
    Head("requête", nom_com, nom_cheflieu),
    Atom("commune", _, _, _, num_dep, _, _, _, _, _, nom_com, _, _),
    Atom("département", num_dep, _, num_cheflieu, _, _, _, nom_dep),
    Atom("commune", _, num_cheflieu, _, _, _, _, _, _, _, nom_cheflieu, _, _)
)
```

Où les variables utilisées sont des strings les identifiant de manière unique. Cette requête est équivalente en SQL à

```sql
SELECT c.nom, cheflieu.nom
FROM commune AS c
JOIN departement AS d ON c.departement = d.id
JOIN commune AS cheflieu ON d.cheflieu = cheflieu.id
```

Une classe `Instance` permet de rassembler les différentes tables (méthode `Instance.add_table`) et d'exécuter des requêtes dessus (méthode `Instance.query`).

## 2.2 Exécution de requêtes

Si l'évaluation d'une requête conjonctive est NP-complète, Yannakakis a introduit en 1981 un algorithme éponyme évaluant une requête conjonctive acyclique en temps linéaire en les tailles de la requête, de l'instance et de la sortie. [2], [3], [4]

Yannakakis

**Entrée :** une requête conjonctive $q$, une instance $I$
**Sortie :** $q(I)$
$J \leftarrow$ instance consistante issue de $I$
$T \leftarrow$ arbre de jointure de $q$ avec une racine choisie arbitrairement
$R_1, ..., R_n \leftarrow$ parcours postfixe de $T$
**Pour** $i = 1, ..., n$ **faire**
  $S_1, ..., S_k$ enfants de $R_i$ dans $T$
  **Pour** $j = 1, ..., k$ **faire**
    $R_i^J \leftarrow \pi_{\text{vars}(R_i) \cup \text{head}(q)} \left( R_i^J \bowtie S_j^J \right)$
  **Fin**
**Fin**

**Algorithme 1:** Yannakakis

Cet algorithme élague le plus tôt possible les tuples pendants (*dangling tuples* en anglais), c'est-à-dire les tuples n'intervenant pas dans le résultat de la requête.

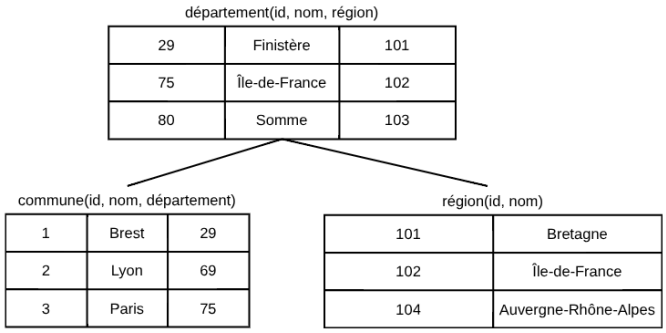



**Abb. 2:** Arbre de jointure avant les passes

Supposons que l'arbre de jointure soit de la forme ci-dessus. La première passe *bottom-up* part des feuilles jusqu'à la racine en éliminant les tuples des parents n'ayant pas d'attribut correspondant dans un de ses enfants. Par exemple le département 80 n'apparaît pas dans la table commune.

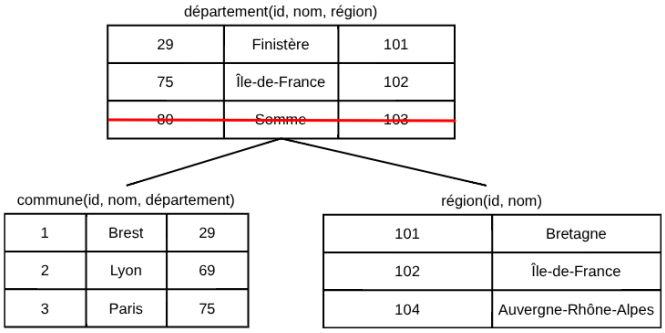



**Abb. 3:** Arbre de jointure après la passe *bottom_up*

La seconde passe *top-down* se fait de la racine vers les feuilles en éliminant cette fois des tuples pendants des enfants. Ici, les communes de département 69 et la région d'identifiant 104 sont éliminées du résultat.

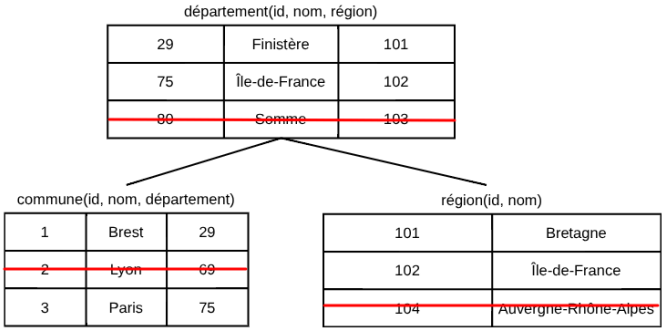



**Abb. 4:** Arbre de jointure après les deux passes

Ces deux passages s'implémentent sous la forme de fonctions récursives `bottom_up` et `top_down`, toutes deux utilisant une fonction auxiliaire `semi_join` réalisant une semi-jointure entre les tables $R$ et $S$, notée $R \ltimes S$. Celle-ci contient les tuples de $R$ pouvant être joints avec au moins un élément de $S$.

$$R \ltimes S = \pi_{\text{vars}(R)}(R \bowtie S)$$

La construction des résultats se fait enfin avec un dernier parcours de la racine vers les feuilles, en réalisant des jointures successives avant de projeter le résultat sur les variables de l'en-tête de la requête. Une fonction `query` orchestre les étapes successives :

- construction de l'arbre de jointure (réduction GYO)
- passage *bottom-up*
- passage *top-down*
- construction des résultats

**Théorème** (Correction de l'algorithme de Yannakakis)**.** *L'algorithme de Yannakakis évalue une requête conjonctive acyclique $q$ en temps polynomial en la taille de $q$, de l'instance $I$ et de la sortie* OUT*.*

**Démonstration.**

- ***Correction****. Dans l'arbre de jointure, si une variable $x$ apparaît dans un nœud, mais pas dans son père, elle ne va apparaître dans le résultat d'aucun des ancêtres ou enfants de ce nœud. Les tuples éliminés ne sont donc pas utiles dans la construction du résultat.*
- ***Complexité****.*
  - *Afin d'obtenir la complexité souhaitée, on montre qu'aucun résultat intermédiaire ne peut être plus grand qu'un polynôme en la taille de l'entrée et de la sortie. On considère un nœud $R_i$ avec pour enfants $S_1, ..., S_n$ dans un arbre de jointure correspondant à une requête.*

- *Ce nœud $R_i$ (nœud dans l'arbre de jointure) sera remplacé par un nœud $R_i^J = \pi_{Y \cup Z}(T)$ où $Y = \text{vars}(R_i)$ et $Z = \text{head}(q) \setminus Y$ et $T = R_i \bowtie S_1 \bowtie \ldots \bowtie S_n$*
- *Puisque $Y$ et $Z$ sont disjoints, $\pi_{Y \sqcup Z}(T) \subset \pi_Y(T) \times \pi_Z(T)$. On remarque que $\pi_Y(T)$ est de taille bornée par celle de $R_i$, et que $\pi_Z(T)$ l'est par la taille de la sortie (car l'exploration se fait dans l'ordre postfixe, donc on a déjà réduit les* dangling tuples *des enfants $S_1$, …, $S_n$.*

## 2.3 Résultats expérimentaux

Voici un exemple de requête qui permet d'obtenir le nom des communes, départements et régions.

```sql
SELECT commune.nom, departement.nom, region.nom FROM commune
JOIN departement ON departement.numero = commune.departement
JOIN region ON region.numero = commune.region
```

Le temps d'exécution de cette requête est de 0.17 secondes, ce qui est de premier abord assez lent.
Toutefois, on remarque que les deux derniers parcours de l'arbre de jointure se font de la racine aux feuilles : on peut en réalité construire le résultat lors de la passe *top-down* directement, ce qui permet de passer à 0.08 secondes pour cette même requête.
De manière générale, cette version « double parcours » réunissant seconde passe et construction du résultat est jusqu'à deux fois plus rapide, ce qui est un gain significatif.

On peut ensuite observer expérimentalement l'évolution du temps d'exécution selon la taille de l'instance $I$, de la sortie OUT et du nombre de jointures dans la requête. Pour cela, j'ai exécuté trois requêtes réalisant respectivement une, deux et trois jointures sur quatre tables, auxquelles je rajoute au fur et à mesure des éléments tirés aléatoirement dans un intervalle pour augmenter la taille de l'instance et du résultat simultanément.
Chaque temps mesuré est la moyenne de cinq mesures successives, avec une incertitude estimée à partir d'elles.

On observe bien une dépendance linéaire en $|I| + |\text{OUT}|$, avec une régression linéaire extrêmement proche des points expérimentaux, ce qui est bien ce qui était attendu. En revanche, pour $I$ contenant quatre millions de tuples (quatre tables d'un million d'éléments chacun), une requête avec trois jointures augmente le temps d'exécution d'une demi-seconde par rapport à une seule jointure, soit de 25% : les requêtes complexes risquent d'avoir une constante très élevée.

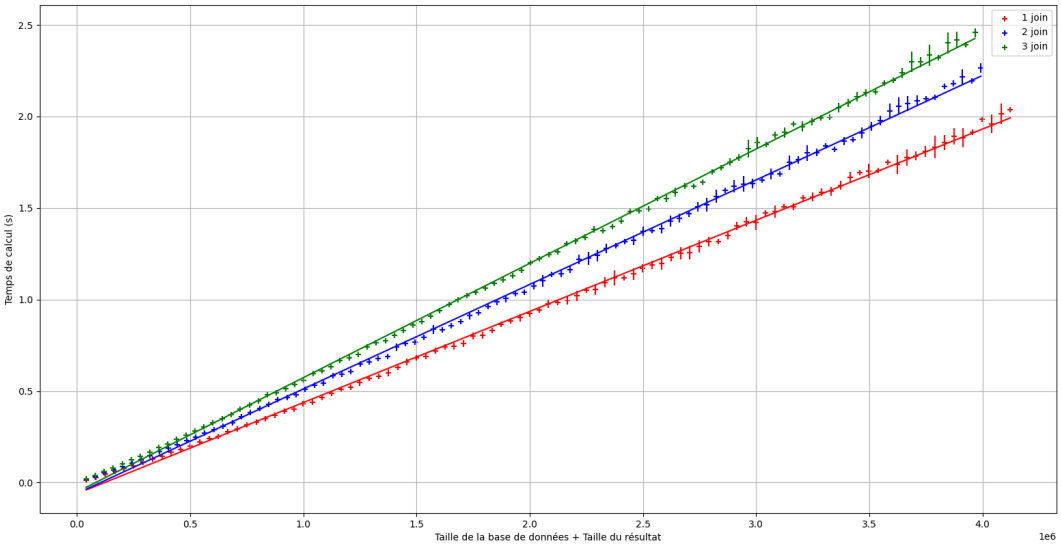



**Abb. 5:** Temps d'exécution en fonction de $|I| + |\text{OUT}|$

Finalement, le temps d'exécution reste très lent comparé à l'implémentation `sqlite3` : il y a un facteur trois entre ces temps pour une instance avec deux millions d'éléments.

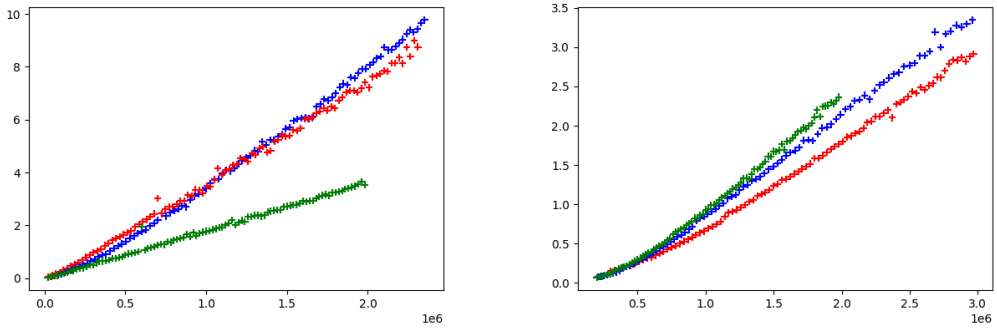



**Abb. 6:** Temps d'exécution de trois requêtes avec mon implémentation (à gauche) et celle de `sqlite3` (droite)

Ce constat est probablement dû à différents facteurs :

- l'implémentation de `sqlite3` est en C, ce qui permet une gestion plus fine de la mémoire
- `sqlite3` n'utilise *a priori* pas le même algorithme pour les différentes requêtes, selon le plan d'exécution choisi. C'est ce qui est sans doute à l'origine des coefficients directeurs très proches entre les trois requêtes
- mon implémentation utilise des `set` pour réaliser les semi-jointures, afin d'identifier les attributs partagés. Dans la pratique, certaines structures bien plus légères sont utilisées, comme les filtres de Bloom, qui garantissent l'absence de faux négatif
- dès qu'il y a de nombreuses jointures, certains choix d'arbres de jointures et de racines sont plus judicieux pour éliminer le plus tôt possible les tuples pendants.

# 3 Conclusion

Ce TIPE a permis de mettre en évidence l'efficacité de l'algorithme de Yannakakis pour le traitement des requêtes conjonctives acycliques. Contrairement à l'approche naïve par produit cartésien, cet algorithme garantit une complexité proportionnelle à la taille des données d'entrée et de sortie, évitant ainsi l'explosion combinatoire des résultats intermédiaires grâce aux passes de réduction.
Si l'implémentation en Python a montré ses limites face à des outils comme SQLite, les tests expérimentaux ont confirmé la linéarité théorique. L'optimisation consistant à fusionner la seconde passe et la construction du résultat a permis un gain de performance de 30% à 50%, illustrant l'importance des choix d'implémentation.
Pour aller plus loin, une réécriture dans un langage plus bas niveau permettrait d'obtenir de meilleures performances. De plus, la réalisation d'une sélection après les jointures ainsi que la généralisation de cet algorithme aux requêtes cycliques, en construisant par exemple un ordre sur les atomes avec un filtre de Bloom, offriraient un point de départ intéressant pour traiter n'importe quelle requête SQL.

# Références

[1] J. Brault-Baron, « Hypergraph Acyclicity Revisited ». [En ligne]. Disponible sur: https://arxiv.org/abs/1403.7076

[2] M. Yannakakis, « Algorithms for Acyclic Database Schemes », in *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7*, 1981. [En ligne]. Disponible sur: https://www.researchgate.net/publication/200034379_Algorithms_for_Acyclic_Database_Schemes

[3] P. Koutris, « Foundations of Data Management : Acyclic Conjunctive Queries ». [En ligne]. Disponible sur: https://pages.cs.wisc.edu/~paris/cs784-f19/lectures/lecture4.pdf

[4] X. Hu, « CS848 Fall 2025: Algorithmic Aspects of Query Processing - Traditional Query Processing ». [En ligne]. Disponible sur: https://cs.uwaterloo.ca/~xiaohu/courses/CS848/CS848-F25-03-Yannakakis-part2.pdf

[5] C. Berge, *Graphes et hypergraphes*. in Dunod université: Mathématiques. Dunod, 1970. [En ligne]. Disponible sur: https://books.google.fr/books?id=OtFbJceS97EC

# 4 Annexe

## 4.1 Hypergraphe

La définition d'une requête conjonctive acyclique nécessite l'introduction d'hypergraphe associé à une requête. On utilisera par la suite la définition donnée par Claude Berge [5].

**Définition** (Hypergraphe)**.** *Étant donné $X$ ensemble fini, un hypergraphe sur $X$ est une famille $H = (E_1, ..., E_p)$ de parties de $X$ vérifiant les points suivants*

- $$\forall i \in [\![1, p]\!], E_i \neq \emptyset$$
- $$\bigcup_{i=1}^{p} E_i = X$$

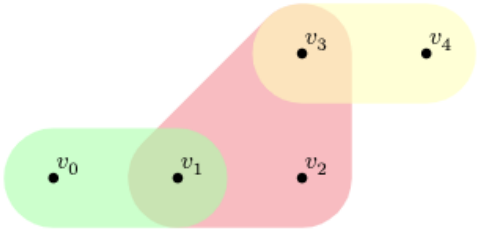



**Abb. 7:** Exemple d'hypergraphe

**Définition** (Arbre de jointure)**.** *Soit $H$ un hypergraphe sur $X$, et $T$ un graphe ayant pour sommet les arêtes de $H$. $T$ est un arbre de jointure si*

- *$T$ est un arbre*
- *pour tout sommet $v$ de $H$, le sous-graphe de $T$ engendré par l'ensemble des arêtes de $H$ contenant $v$ est connexe*

*Un tel hypergraphe est dit $\alpha$-acyclique.*

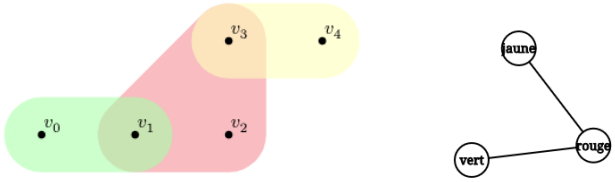



**Abb. 8:** Hypergraphe et un arbre de jointure associé

## 4.2 Acyclicité

Un hypergraphe est dit $\alpha$-acyclique si et seulement s'il peut être réduit à l'hypergraphe vide par l'algorithme GYO.

Graham-Yu-Ozsoyoglu

**Entrée :** hypergraphe $H$
**Sortie :** booléen indiquant si $H$ est acyclique

**Tant que** $H$ possède un sommet isolé ou une oreille **faire**
  **Si** $v$ est un sommet isolé (appartient à une seule arête $E$) **alors**
    retirer $v$ de $E$ et de $X$
  **Si** $E$ est une oreille (incluse dans une autre arête $F$) **alors**
    retirer $E$ de $H$
**Fin**

**Renvoyer** Vrai si $H$ est vide, Faux sinon

**Algorithme 2:** Graham-Yu-Ozsoyoglu

**Théorème** (Équivalence de l'acyclicité)**.** *Soit $H$ un hypergraphe. Les propriétés suivantes sont équivalentes :*

1. *$H$ est $\alpha$-acyclique (réductible à vide par l'algorithme GYO).*
2. *$H$ possède un arbre de jointure*

**Démonstration.**

- **1 ⇒ 2** *: On procède par récurrence sur le nombre d'arêtes $n$.*
  - *Pour $n = 1$, l'arbre trivial à un seul nœud est un arbre de jointure.*
  - *Supposons la propriété vraie pour $n - 1$.*
    - *Si $H$ est réductible, il existe une oreille $E$ incluse dans une arête $F$ de $H \setminus \{E\}$. Par hypothèse de récurrence, $H \setminus \{E\}$ possède un arbre de jointure $T'$.*
    - *En ajoutant le nœud $E$ et l'arête $(E, F)$ dans $T'$, on obtient un arbre $T$.*
    - *Vérifions la connexité : pour toute variable $x \in E$, par définition de l'oreille, on a $x \in F$. Si $x$ appartient à une autre arête $G$ de $H$, comme l'ensemble des nœuds contenant $x$ dans $T'$ est connexe et $T'$ contient $F$ et $G$, et que $E$ est relié à $F$, $E$ est également relié à $G$. L'ensemble des nœuds de $T$ contenant $x$ dans $T$ est donc connexe.*
- **2 ⇒ 1** ***(Existence d'une oreille)*** *:*
  - *Si $H$ possède un arbre de jointure $T$, considérons une feuille $E$ de $T$ et $F$ son unique voisin. Par la propriété de connexité, toute variable $x$ de $E$ présente dans une autre arête $G$ doit être présente sur le chemin unique entre $E$ et $G$ dans l'arbre. Ce chemin passe nécessairement par $F$. Ainsi, $\left(E \cap \bigcup_{G \neq E} G\right) \subset F$. $E$ est donc une oreille éliminable par GYO.*
  - *Par itération, l'hypergraphe est entièrement réductible.*

La validité de cet algorithme repose sur deux propriétés critiques :

1. **Terminaison** : À chaque étape, on retire soit un sommet, soit une arête. Comme l'hypergraphe est fini, l'algorithme s'arrête en au plus $|X| + |H|$ étapes.
2. **Influence de l'ordre** : L'ordre dans lequel on retire les oreilles ou les sommets isolés n'influence pas le résultat final. Si un hypergraphe est réductible, n'importe quelle séquence de réductions valides mènera à l'hypergraphe vide.

## 4.3 Construction de l'arbre de jointure

L'arbre de jointure se construit au cours de la réduction GYO :

1. Chaque atome de la requête devient un nœud de l'arbre.

2. Lorsqu'une arête $E$ est supprimée, car elle est incluse dans une arête $F$, on ajoute une arête entre les nœuds correspondant à $E$ et $F$ dans l'arbre.
3. Si plusieurs arêtes sont candidates pour $F$, n'importe laquelle convient pour maintenir la connexité des variables communes.

Cette structure garantit la propriété de connexité : pour toute variable $x$, l'ensemble des nœuds contenant $x$ forme un sous-graphe connexe (un arbre), ce qui est la condition nécessaire pour que les passes de semi-jointures propagent correctement les contraintes.

## 4.4 Code

### 4.4.1 Hypergraphe

```py
from typing import List

class Arete:
    cardinal: int
    field: int
    indice: int

    """
    Arête d'un hypergraphe.
    - sommets : bitfield des éléments des sommets
    - cardinal : nombre d'éléments dans sommets
    """
    def __init__(self):
        self.cardinal = 0
        self.field = 0
        self.indice = -1

    @property
    def sommets(self):
        sommets = []
        field = self.field

        i = 0
        while field != 0:
            if field & 1 == 1:
                sommets.append(i)
            i += 1
            field >>= 1

        return sommets

    def copier(self):
        a = Arete()
        a.field = self.field
        a.cardinal = self.cardinal
        a.indice = self.indice
        return a

```

```
39      def augmenter(self, *sommets: int):
40          for s in sommets:
41              if not self.contient(s):
42                  self.field |= 1 << s
43                  self.cardinal += 1
44
45      def reduire(self, sommet: int):
46          if self.contient(sommet):
47              self.field &= ~(1 << sommet)
48              self.cardinal -= 1
49
50      def dans(self, a):
51          return (self.field | a.field) == a.field
52
53      def contient(self, sommet: int):
54          return self.field & (1 << sommet) != 0
55
56      def __repr__(self):
57          return f"Arete(sommets={self.sommets}, indice={self.indice})"
58
59
60  class Hyper:
61      aretes: List[Arete]
62      ordre: int
63
64      """
65      Hypergraphe.
66      - ordre : nombre de sommets (numérotés 0..ordre-1)
67      - aretes : liste d'objets Arete
68      """
69      def __init__(self, ordre: int):
70          self.ordre = int(ordre)
71          self.aretes = []
72
73      @property
74      def nb_aretes(self):
75          return len(self.aretes)
76
77      def decompte(self, sommet: int):
78          return sum((1 if a.contient(sommet) else 0) for a in self.aretes)
79
80      def est_valide(self):
81          """
82          Vérifie que chaque arête a cardinal > 0 et que chaque sommet de 0..ordre-1
83          apparaît dans au moins une arête.
84          """
85          if any(a.cardinal == 0 for a in self.aretes):
86              return False
```

```
87
88          couvert = 0
89          for a in self.aretes:
90              couvert |= a.field
91
92          while couvert != 0:
93              if couvert & 1 == 0:
94                  return False
95              couvert = couvert >> 1
96
97          return True
98
99      def retirer_sommet(self, sommet: int):
100         for a in self.aretes:
101             a.reduire(sommet)
102
103     def copier(self):
104         h = Hyper(self.ordre)
105         h.aretes = [a.copier() for a in self.aretes]
106         return h
107
108     def est_acyclique(self):
109         h = self.copier()
110
111         restant = set(range(h.ordre))
112         sommets_compte = [h.decompte(s) for s in range(h.ordre)]
113
114         retirer = []
115         while True:
116             sortir = True
117
118             # On enlève tous les sommets dans moins d'une arête
119             retirer.clear()
120             for s in restant:
121                 if sommets_compte[s] <= 1:
122                     retirer.append(s)
123                     sortir = False
124             for s in retirer:
125                 restant.discard(s)
126                 h.retirer_sommet(s)
127
128             # On enlève toutes les arêtes inclues dans une autre
129             for indice, a in enumerate(h.aretes):
130                 if a.cardinal > 0:
131                     for b in h.aretes:
132                         if a.dans(b) and a.indice != b.indice:
133                             for s in a.sommets:
134                                 sommets_compte[s] -= 1
```

```
                            h.enlever_arete(indice)
                            sortir = False
                            break

            # On continue éventuellement l'algorithme
            if sortir or len(restant) == 0:
                break

        return len(restant) == 0

    # Renvoie l'arbre si la requête est acyclique, None sinon
    def arbre_jointure(self):
        arbre = [[] for _ in range(self.nb_aretes)]

        h = self.copier()

        restant = set(range(h.ordre))
        sommets_compte = [h.decompte(s) for s in range(h.ordre)]

        retirer = []
        while True:
            sortir = True

            # On enlève tous les sommets dans moins d'une arête
            retirer.clear()
            for s in restant:
                if sommets_compte[s] <= 1:
                    retirer.append(s)
                    sortir = False
            for s in retirer:
                restant.discard(s)
                h.retirer_sommet(s)

            # On enlève toutes les arêtes inclues dans une autre
            for indice, a in enumerate(h.aretes):
                if a.cardinal > 0:
                    for b in h.aretes:
                        if a.dans(b) and a.indice != b.indice:
                            arbre[a.indice].append(b.indice)
                            arbre[b.indice].append(a.indice)
                            for s in a.sommets:
                                sommets_compte[s] -= 1
                            h.enlever_arete(indice)
                            sortir = False
                            break

            # On continue éventuellement l'algorithme
            if sortir or len(restant) == 0:
```

```
183                 break
184
185         return arbre if len(restant) == 0 else None
186
187     def ajouter_arete(self, *aretes: Arete):
188         for a in aretes:
189             a.indice = len(self.aretes)
190             self.aretes.append(a)
191
192     def enlever_arete(self, indice: int):
193         del self.aretes[indice]
194
195     def __repr__(self):
196         return f"Hyper(ordre={self.ordre}, nb_aretes={self.nb_aretes},
            aretes={self.aretes})"
```

### 4.4.2 Table

```py
from csv import reader
from typing import Dict, List, Self, Set

class Table:
    name: str  # Nom de la table
    rows: List[tuple]  # Lignes

    columns: List[str]  # En-tête

    def __init__(self, name):
        self.name = name
        self.rows = []
        self.columns = []

    @property
    def len(self):
        return len(self.rows)

    # Mise à jour du nom de colonne
    def rename(self, index: int, nname: str):
        self.columns[index] = nname

    # Importation d'un fichier csv
    def import_from(self, path: str):
        self.rows = []

        with open(path, mode="r") as file:
            csv = reader(file)

            self.columns = csv.__next__()
            self.rows.extend([tuple(line) for line in csv])
```

```
32
33      # Projection et sélection
34      def project(self, selection: List[bool], validate):
35          switch = []
36
37          for i, b in enumerate(selection):
38              if b:
39                  switch.append(i)
40
41          dedup = set()
42          for row in self.rows:
43              if validate(row):
44                  dedup.add(tuple(row[i] for i in switch))
45
46          self.columns = [self.columns[i] for i in switch]
47          self.rows = list(dedup)
48
49      def __repr__(self):
50          return f"Table(labels={self.columns}, rows_count={len(self.rows)})"
51
52      def print(self):
53          print("Labels ", self.columns)
54          print(" - " + "\n - ".join(str(r) for r in self.rows))
```

### 4.4.3 Requête

```py
1   from typing import List
2
3   class Atom:
4       name: str                   # Nom de la table de l'atome
5       index: int                  # Indice de l'atome
6       variables: set[str]         # Variables présentes dans l'atome
7
8       original: tuple[str]
9
10      def __init__(self, name, *variables):
11          self.name = name
12          self.variables = set(variables)
13          self.record = { v:i for i, v in enumerate(variables) }
14          self.original = variables
15
16          self.variables.discard('_')
17
18      def __repr__(self):
19          return f"{self.name}({ ', '.join(str(v) for v in self.original) })"
20
21  class Head:
22      name: str               # Nom de la tête pour faire joli
23      variables: tuple[str]   # Variables présentes dans la tête
```

```
24
25      def __init__(self, name: str, *variables):
26          self.name = name
27          self.variables = variables
28
29      def __repr__(self):
30          return f"{self.name}({ ', '.join(str(v) for v in self.variables) })"
31
32  class Query:
33      atoms: tuple[Atom]      # Atomes de la requête
34      head: Head              # Tête de la requête
35
36      variables: set[str]     # Variables présentes dans la requête
37
38      def __init__(self, head: Head, *atoms):
39          self.head = head
40          self.atoms = atoms
41          self.variables = set()
42
43          for i, atom in enumerate(self.atoms):
44              atom.index = i
45              self.variables = self.variables.union(atom.variables)
46
47      def __repr__(self):
48          return f"{self.head} :- {', '.join(str(a) for a in self.atoms)}"
```

### 4.4.4 Instance

```py
1   import time
2   from collections import defaultdict
3   from csv import reader
4   from typing import Dict, List, Self, Set
5
6   from table import Table
7   from hyper import Arete, Hyper
8   from request import Atom, Head, Query
9
10  class JoinNode:
11      atom: Atom              # Atome du noeud
12      table: Table            # Table correspondant à l'atome
13
14      record: Dict[str, int]  # Annuaire de la table du noeud
15      variables: Set[str]          # Variables pertinentes du noeud
16
17      children: List[Self]    # Enfants dans l'arbre de jointure
18      root: bool    # Parent dans l'arbre de jointure
19
20      def __init__(self, atom: Atom, table: Table):
21          self.atom = atom
```

```
22          self.table = Table(table.name)
23
24          self.record = dict()
25          self.variables = set()
26
27          self.children = []
28          self.root = False
29
30          self.table.columns = table.columns.copy()
31          self.table.rows = table.rows
32
33          for i, v in enumerate(atom.original):
34              if v != '_':
35                  self.variables.add(v)
36                  self.record[v] = i
37                  self.table.columns[i] = v
38              else:
39                  self.table.columns[i] = '_'
40
41      # Pour ajouter un enfant
42      def push_child(self, child: Self):
43          self.children.append(child)
44
45      # Semi-jointure
46      def semi_join(self, node: Self):
47          intersection = sorted(self.variables.intersection(node.variables))
48
49          index = set(
50              tuple(row[node.record[v]] for v in intersection) for row in node.table.rows
51          )
52
53          self.table.project(
54              [v in self.variables for v in self.table.columns],
55              lambda row: tuple(row[self.record[v]] for v in intersection) in index
56          )
57
58          # Mise à jour des colonnes (ne garder que les variables, pas les "_")
59          self.record = {v: i for i, v in enumerate(self.table.columns) if v != '_'}
60
61      def patch_rows(self, row_a, row_b, selection: list[tuple[bool, int]]):
62          return tuple(row_a[i] if a else row_b[i] for a, i in selection)
63
64      # Jointure complète
65      def full_join(self, node: Self, head: Head, strict):
66          intersection = sorted(self.variables.intersection(node.variables))
67          index = defaultdict(list)
68
69          # Construction de la sélection
```

```
70          selection = []
71          columns = []
72          vars_iter = head.variables if strict else
            sorted(self.variables.union(node.variables))
73          for v in vars_iter:
74              if v in self.variables or v in node.variables:
75                  current = v in self.variables
76                  columns.append(v)
77                  selection.append((current, self.record[v] if current else
                    node.record[v]))
78              else:
79                  raise ValueError(f"Variable {v} non présente lors de la jointure.")
80
81          # Construction du résultat
82          for row in node.table.rows:
83              key = tuple(row[node.record[v]] for v in intersection)
84              index[key].append(row)
85
86          result = []
87          for row in self.table.rows:
88              key = tuple(row[self.record[v]] for v in intersection)
89
90              if key in index:
91                  for row2 in index[key]:
92                      result.append(self.patch_rows(row, row2, selection))
93
94          self.table.rows = result
95          self.table.columns = columns
96          self.record = {v: i for i, v in enumerate(columns)}
97          self.variables = set(head.variables) if strict else
            set(self.variables.union(node.variables))
98
99
100     def compute(self, head: Head):
101         # Cas d'une racine sans enfants (ou requête à un seul atome)
102         if self.root and not self.children:
103             self.table.project(
104                 [v in head.variables for v in self.table.columns],
105                 lambda row: True
106             )
107             self.record = { v:i for i, v in enumerate(self.table.columns) }
108             self.variables = set(head.variables)
109             return
110
111         for index, child in enumerate(self.children):
112             child.compute(head)
113             self.full_join(child, head, self.root and index == len(self.children) - 1)
114
115
```

```python
class Instance:
    tables: dict[str, Table]  # Annuaire nom-table

    def __init__(self):
        self.tables = dict()

    def add_table(self, table: Table):
        self.tables[table.name] = table

    # Crée l'arbre de jointure d'une requête
    def join_dfs(self, tree, query: Query, seen, i):
        seen.add(i)
        jointure = JoinNode(query.atoms[i], self.tables[query.atoms[i].name])

        for j in tree[i]:
            if j not in seen:
                jointure.push_child(self.join_dfs(tree, query, seen, j))

        return jointure

    def jointure(self, query: Query):
        hyper = Hyper(len(query.variables))

        record = dict()
        #record_bis = dict()

        for i, v in enumerate(query.variables):
            record[v] = i
            #record_bis[i] = v

        for atom in query.atoms:
            arete = Arete()
            arete.augmenter(*[record[v] for v in atom.variables])
            hyper.ajouter_arete(arete)

        arbre = hyper.arbre_jointure()

        if not arbre:
            raise ValueError("La requête est cyclique (non supporté par l'algorithme de Yannakakis).")

        #arbre_bis = dict()

        #for i, adj in enumerate(arbre):
        #    arbre_bis[record_bis[i]] = [record_bis[j] for j in adj]

        vus = set()
        return self.join_dfs(arbre, query, vus, 0)

```

```python
    # Passage bottom -> up (Remontée des contraintes vers la racine)
    def bottom_up(self, query: Query, node: JoinNode):
        for child in node.children:
            self.bottom_up(query, child)

            variables = node.variables.intersection(child.variables)
            if len(variables) > 0:
                node.semi_join(child) # Élimination des tuples du parent sans correspondance (bottom-up)

    # Passage top -> down (Propagation des contraintes vers les feuilles)
    def top_down(self, query: Query, node: JoinNode):
        for child in node.children:
            variables = node.variables.intersection(child.variables)

            if len(variables) > 0:
                child.semi_join(node) # Propagation de la réduction vers les feuilles (top-down)
            self.top_down(query, child)

    # Réalisation d'une requête
    def query(self, query: Query):
        # Vérification de la requête
        for atom in query.atoms:
            if atom.name not in self.tables:
                raise ValueError(f"Table '{atom.name}' non présente dans l'instance.")
            if len(atom.original) != len(self.tables[atom.name].columns):
                raise ValueError(
                    f"Nombre de colonnes incorrect pour l'atome {atom.name}"
                )

        # Création de l'arbre de jointure
        jointree = self.jointure(query)
        jointree.root = True

        # Premier passage
        self.bottom_up(query, jointree)

        # Deuxième passage
        # Suit directement l'algorithme de Yannakakis, mais il est possible de sauter cette étape
        self.top_down(query, jointree)

        # On réalise la jointure
        jointree.compute(query.head)

        return jointree.table
```